CS683 Programming Assignment 1: Task 2
Matrix Multiplication for Inference

Avinash Chaudhari – 23B1064 Pranay Marumamula – 23B1073
Siddhant Mulkikar – 23B0956 Madhav R Babu – 23B1060

Autumn 2026

1 System Configuration

- **CPU:** Intel Core Ultra 5 125U (Meteor Lake), hybrid architecture: 2 P-cores (4 threads with HT, max 4.30 GHz) + 8 E-cores (max 3.60 GHz) + 2 LP E-cores (max 2.10 GHz), 14 logical CPUs total
- **L1-D Cache (P-core):** 48 KiB per core, 64-byte lines, 12-way set associative
- **L2 Cache (P-core):** 2 MiB per core, 16-way
- **L3 Cache:** 12 MiB (1 instance, shared), 12-way
- **Vector ISA:** AVX2 + FMA, AVX-VNNI (no AVX-512)
- **Compiler flags:** `-std=c++17 -O2 -fno-tree-vectorize -mavx2 -mfma`
- **OS:** Ubuntu 26.04.1 LTS, kernel 7.0.0-31-generic (x86_64)
- **Pinning:** `taskset -c 3` (a P-core). On this hybrid CPU, `perf` reports both `cpu_core/` and `cpu_atom/` events; all hardware counter measurements use the `cpu_core/` prefix.
- **Frequency governor:** `powersave`, with `kernel.perf_event_paranoid = -1` so `perf` can read the hardware counters used for cache miss and instruction count measurements.

2 Task 2A: Software Prefetching (5 points)

2.1 Overview

Software prefetching issues explicit cache-line fetch instructions ahead of demand loads, hiding memory latency by overlapping data transfer with computation. We implement the prefetch stage as a **scalar** (non-SIMD) kernel with five prefetch modes, controlled by compile-time defines:

- **Mode 0 (none)**: baseline: no prefetch instructions.
- **Mode 1 (within_row)**: prefetches A and B elements `PF_DIST` floats ahead within the current dot-product loop, once per cache line (every 16 floats).
- **Mode 2 (next_B)**: prefetches the start of the *next* B row before computing the current dot product, so the next column's data is already in cache.
- **Mode 3 (C_output)**: prefetches the C output location before the dot product.
- **Mode 4 (all)**: combines modes 1, 2, and 3.

Two parameters are tuned:

- **Prefetch distance (`PF_DIST`)**: how many floats ahead to prefetch (0, 16, 32, 64, 128).
- **Cache fill level (`PF_HINT`)**: `_MM_HINT_T0` (L1), `T1` (L2), `T2` (L3), `NTA` (non-temporal).

2.2 Impact of Matrix Size

Table 1: Prefetch speedup vs. matrix size (no-prefetch baseline, and best mode at each size)

Matrix Size	Naive GFLOP/s	Best Prefetch GFLOP/s	Best Speedup
128	4.93	5.21	1.03x
256	4.64	5.01	1.07x
512	4.40	4.53	1.01x
1024	4.21	4.33	1.03x
2048	2.64	2.80	1.06x

All measurements in this section were collected with **hardware prefetchers disabled** (MSR `0x1A4` = `0xF`) to isolate the effect of software prefetching alone.

Software prefetching provides only marginal benefit on this scalar kernel (<1.07× at all sizes). The scalar triple loop is *compute-bound*: each output element requires K multiply-accumulate iterations processed one float at a time, and the CPU's out-of-order engine can overlap demand misses with the long instruction stream. Adding explicit prefetch instructions introduces overhead (extra μops competing for execution ports) that nearly cancels any latency-hiding benefit.

2.3 CPU Metrics Analysis

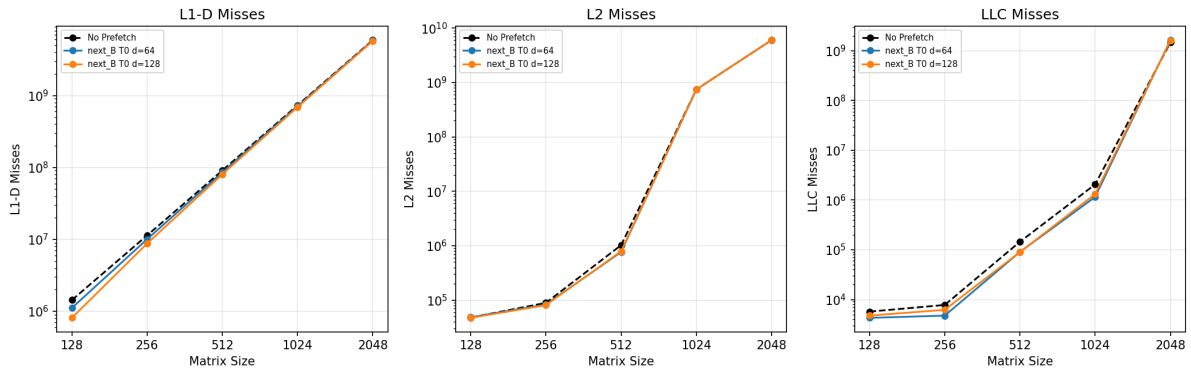


Figure 1: L1-D, L2, and LLC misses: no prefetch vs. next_B mode (T0, distances 64 and 128)

Key observations from the cache miss data:

- **L1-D misses decrease slightly** with next_B prefetching (e.g. at 1024: 732M → 690M for dist=128), because the prefetch pulls the next B row into L1 before demand.
- **L2 misses are unchanged** (∼745M at 1024) because the working set far exceeds L2 regardless of prefetching.
- **LLC misses are similar** across configurations, confirming that the bottleneck is DRAM bandwidth, not cache miss latency.
- **Within-row prefetching (mode 1) increases instruction count dramatically** (681M+ `sw_prefetch_access` at 1024) without reducing misses, because the prefetch instructions consume execution-port bandwidth that the compute loop needs.

2.4 Execution Time and Speedup

2.4.1 Speedup by Prefetch Mode

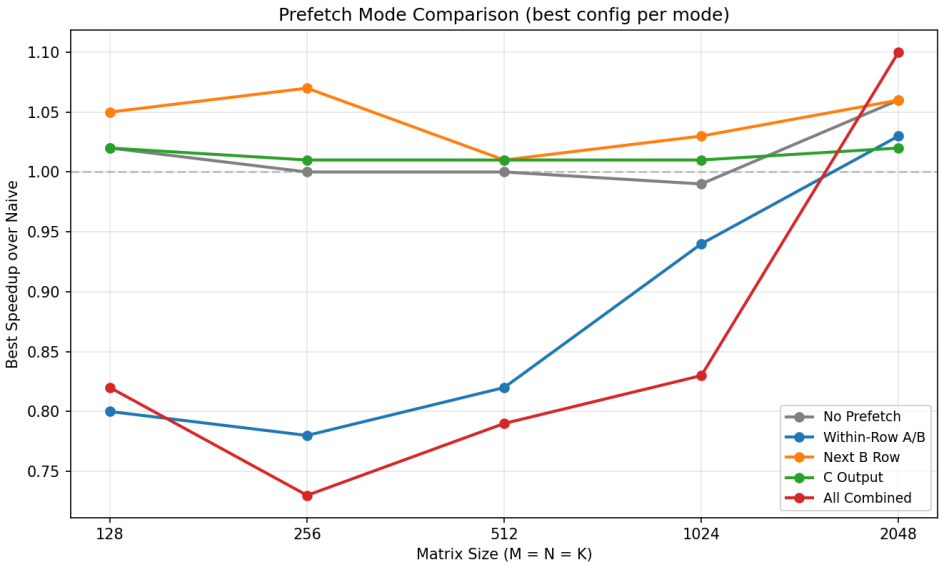


Figure 2: Best speedup per prefetch mode across matrix sizes

next_B is the only mode that achieves speedup $\geq$1.0$\times$ at every size (best configs 1.01–1.07$\times$). **within_row** and **all** lose even at their *best* configurations for sizes up to 1024 (0.73–0.94$\times$), and their typical configurations are far worse (0.42–0.82$\times$ with T0, collapsing to 0.13–0.17$\times$ with NTA): a prefetch every 16 iterations of the inner K loop adds instruction overhead that overwhelms any benefit. At 2048 their best configs scrape just past parity (1.03$\times$ and 1.10$\times$), which is within the naive baseline's run-to-run variance. **C_output** is neutral: the single output store per dot product is too infrequent to benefit from prefetching.

2.4.2 Speedup by Prefetch Distance

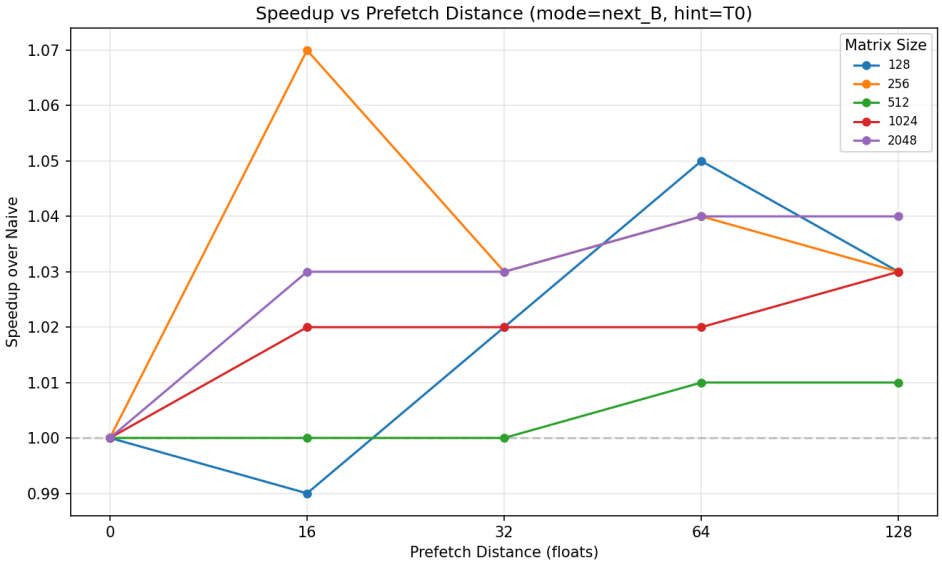


Figure 3: Speedup vs. prefetch distance for next_B mode (T0 hint)

The optimal prefetch distance is **64–128 floats** (4–8 cache lines ahead). At distance 0, the prefetch fires too late to hide any latency. At distances 16–128, the small improvement (∼1.02–1.05×) is consistent across sizes, with 128 slightly better at large matrices where the next B row is more likely to be evicted.

2.4.3 Speedup by Cache Fill Level

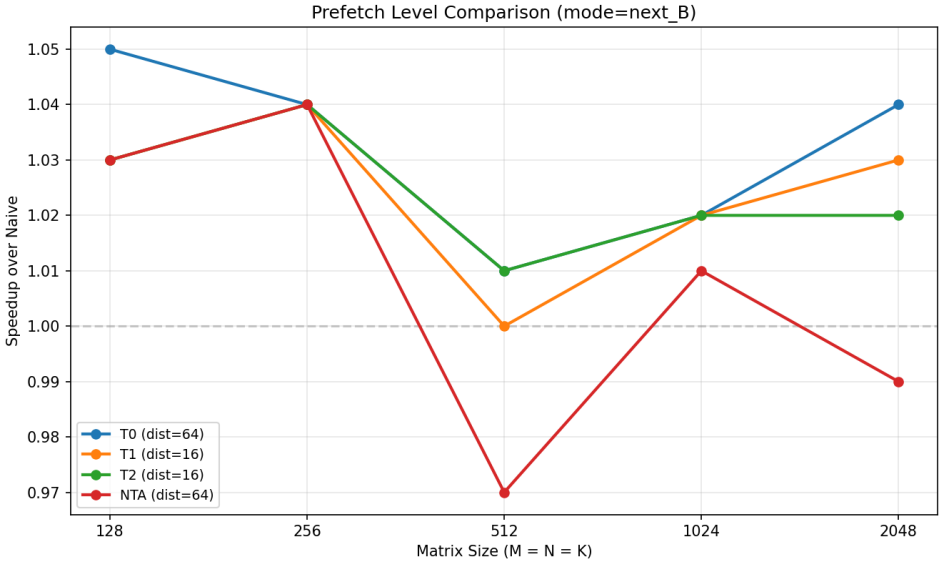


Figure 4: Speedup vs. matrix size for different cache fill levels (next_B mode)

T0 (prefetch into L1) is the best hint for next_B mode. T1 and T2 are slightly worse because prefetching into L2/L3 still requires a demand miss to pull the line into L1 when the dot product actually uses it. NTA (non-temporal) performs similarly to T0 at large sizes but offers no advantage.

2.5 Optimal Parameters

- **Best prefetch mode**: next_B (mode 2): prefetch the start of the next B row.
- **Best prefetch distance**: 64–128 floats.
- **Best cache fill level**: `_MM_HINT_T0` (L1).

However, even the best configuration achieves only ∼1.03–1.06× speedup, even with hardware prefetchers disabled. The scalar kernel is compute-bound: the CPU spends most cycles retiring scalar multiply-accumulate instructions, not stalling on cache misses, so there is little memory latency for software prefetching to hide.

2.6 Effect of Hardware Prefetchers

To isolate the effect of the hardware prefetchers, we ran the naive scalar kernel with all four hardware prefetchers enabled (MSR `0x1A4 = 0x0`) and disabled (MSR `0x1A4 = 0xF`) across five matrix sizes.

Table 2: Naive kernel: hardware prefetchers ON vs. OFF

	Time (ms)		GFLOP/s		L1-D Misses		L2 Misses		LLC Misses	
Size	ON	OFF	ON	OFF	ON	OFF	ON	OFF	ON	OFF
128	1.97	1.08	2.13	3.87	70K	1.4M	84K	52K	4K	6K
256	16.78	10.61	2.00	3.16	286K	11.4M	189K	111K	5K	15K
512	58.62	60.74	4.58	4.42	2.9M	90.9M	1.3M	1.0M	10K	163K
1024	491.36	508.48	4.37	4.22	42.5M	732.2M	745.6M	747.4M	105K	1.1M
2048	5,217	9,418	3.29	1.82	218.4M	5,897M	6,250M	6,030M	33.9M	1,188M

At small sizes (128–256), the naive kernel is actually *faster* with hardware prefetchers **off**. At these sizes the whole working set fits in L2/L3, so an L1 miss costs almost nothing: there is no memory latency for the prefetchers to hide. They still fetch speculatively, however, and this useless extra traffic (visible as *higher* L2 misses with prefetchers on: 84K vs. 52K at 128) gets in the way of the real loads. The runs are also very short (1–17 ms), so part of the gap is simply measurement noise. The prefetchers only start to matter at size 2048, below.

At size 512–1024, the two configurations are nearly identical (∼4.4–4.6 GFLOP/s), since the working set spans L2 and the hardware prefetcher's benefit roughly cancels its overhead.

The hardware prefetcher becomes essential at **size 2048**: with prefetchers on, the naive kernel achieves 3.29 GFLOP/s in 5.2 seconds; with prefetchers off, it drops to 1.82 GFLOP/s and takes 9.4 seconds, a 1.81× slowdown. LLC misses explode from 34M to 1.19B without prefetching, confirming that at this size the working set far exceeds L3 and the hardware streamer is critical for maintaining any memory-level parallelism.

2.7 Analysis Questions

2.7.1 Q1: Speedup Trend with Matrix Size

Prefetch speedup is nearly flat across matrix sizes, staying in the 1.00–1.06× range for the best mode (next_B), even with hardware prefetchers disabled. The scalar kernel is compute-bound at all sizes: at small sizes the data fits in cache so there are few misses to hide; at large sizes the out-of-order engine overlaps the long scalar instruction stream with demand misses, leaving little exposed latency for software prefetch to cover.

2.7.2 Q2: Best Prefetch Distance

The best prefetch distance is **64–128 floats** (256–512 bytes, or 4–8 cache lines). This is enough lead time for the prefetch to complete before the demand load, without being so far ahead that the prefetched line is evicted before use. At distance 0, the prefetch is useless (the demand load fires simultaneously). Distances 16–32 are marginally worse because the prefetch may not complete in time for the next iteration.

2.7.3 Q3: Best Cache Fill Level

T0 (`_MM_HINT_T0`, prefetch into L1) achieves the maximum speedup. This makes sense because the dot-product loop immediately reads the prefetched data, so placing it in L1 avoids one additional cache-to-cache transfer. T1/T2 prefetch into L2/L3, requiring a subsequent L1 miss on the demand load, which partially negates the latency hiding. NTA is designed for streaming data that won't be reused, but each B row is accessed only once per C element anyway, so NTA offers no structural advantage here.

3 Task 2B: SIMD (1 point)

3.1 SIMD Implementation

The SGEMM kernel uses a **register-tiled vectorized dot product** approach. The matrix layout stores both A and B with the reduction axis K as the contiguous (unit-stride) dimension: $C[i][j] = \sum_p A[i][p] \cdot B[j][p]$. This means the natural SIMD strategy is to vectorize the dot product over p.

The micro-kernel computes a 4×2 block of C at once using 8 independent `__m256` accumulators:

- For each step along K, two `_mm256_loadu_ps` loads fetch 8 elements from two rows of B.
- Four `_mm256_broadcast_ss` calls broadcast one element from each of four A rows.
- Eight `_mm256_fmadd_ps` instructions accumulate all $4 \times 2 = 8$ partial-product vectors.

This yields 8 FMAs per step from only 6 loads, and the 8 independent accumulator chains saturate the FMA pipeline (FMA latency $\sim$4 cycles, throughput 2/cycle, requiring $\geq$8 independent chains). After the K loop, each accumulator is horizontally reduced (`hsum256`) to a single scalar, stored into the corresponding $C[i][j]$.

Leftover rows (M mod 4), columns (N mod 2), and K elements (K mod 8) fall to a scalar cleanup path.

Three SIMD widths were benchmarked:

- **SSE 128-bit**: 4 floats per vector, 4×2 register tile, 8 accumulators
- **AVX2 256-bit**: 8 floats per vector, 4×2 register tile, 8 accumulators
- **AVX-512 512-bit**: 16 floats per vector, 4×2 register tile, 8 accumulators

None of these include cache blocking: the kernel streams through A and B from whatever cache level holds them.

3.2 Instruction Count and Performance

Table 3: Instruction count and speedup for different SIMD widths

		Matrix Size ($M = N = K$)				
Metrics		128	256	512	1024	2048
No SIMD	Instructions	94,691,884	721,951,242	5,700,178,491	45,378,549,503	362,206,124,379
	GFLOP/s	3.33	2.64	2.41	2.30	2.21
SSE 128	Instructions	101,691,574	773,677,911	6,098,319,894	48,498,010,433	386,920,963,092
	GFLOP/s	46.48	50.75	51.34	50.52	35.68
	Speedup	13.94x	19.20x	21.28x	22.00x	16.13x
AVX2 256	Instructions	99,048,350	751,129,706	5,912,668,975	46,990,364,336	374,781,905,812
	GFLOP/s	89.68	87.95	90.90	82.41	46.18
	Speedup	26.90x	33.27x	37.67x	35.89x	20.87x
AVX-512	Instructions	97,792,306	740,383,994	5,821,928,574	46,246,355,457	368,749,447,552
	GFLOP/s	89.46	108.57	112.14	99.28	52.09
	Speedup	26.84x	41.07x	46.47x	43.24x	23.54x

Instruction count observation: SIMD does *not* significantly reduce instruction count: the SIMD versions issue roughly the same number of instructions as the naive version (within $\sim$7%). This is because the register-tiled micro-kernel still executes $O(MNK)$ FMA instructions; SIMD packs 4/8/16 multiply-adds into each instruction but also adds overhead from broadcasts, horizontal reductions, and cleanup. The speedup comes not from fewer instructions but from each instruction doing $4\times/8\times/16\times$ more work.

3.3 Plots

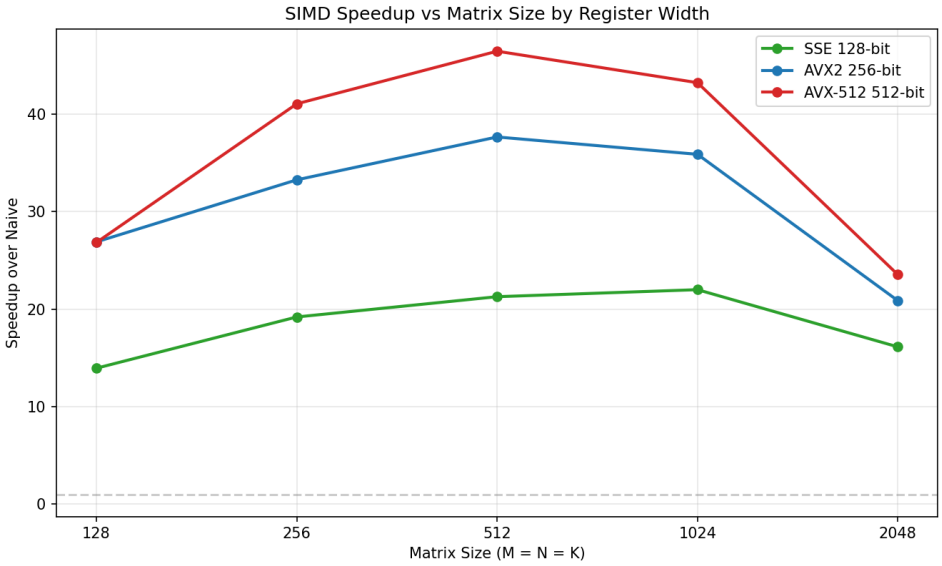


Figure 5: SIMD speedup vs. matrix size for different register widths

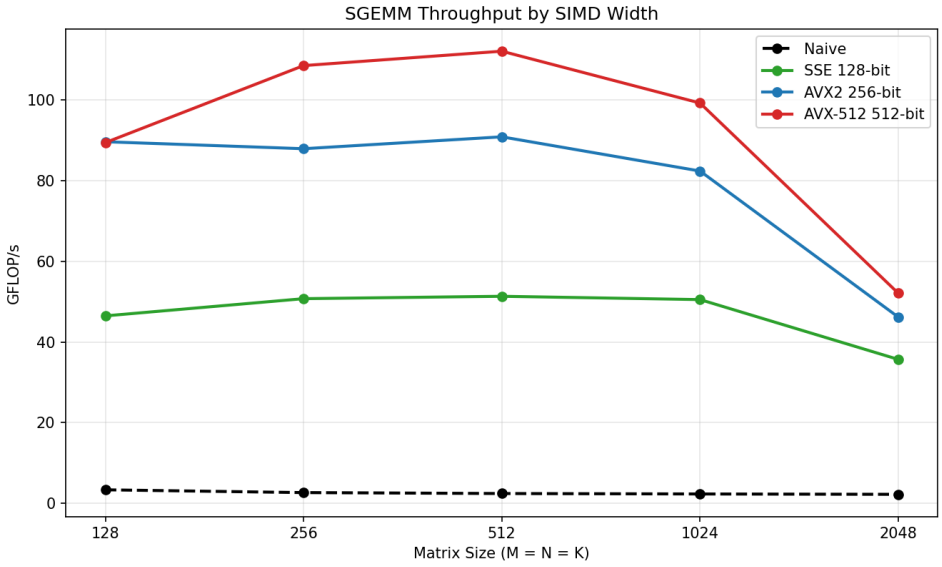


Figure 6: SGEMM throughput (GFLOP/s) by SIMD width

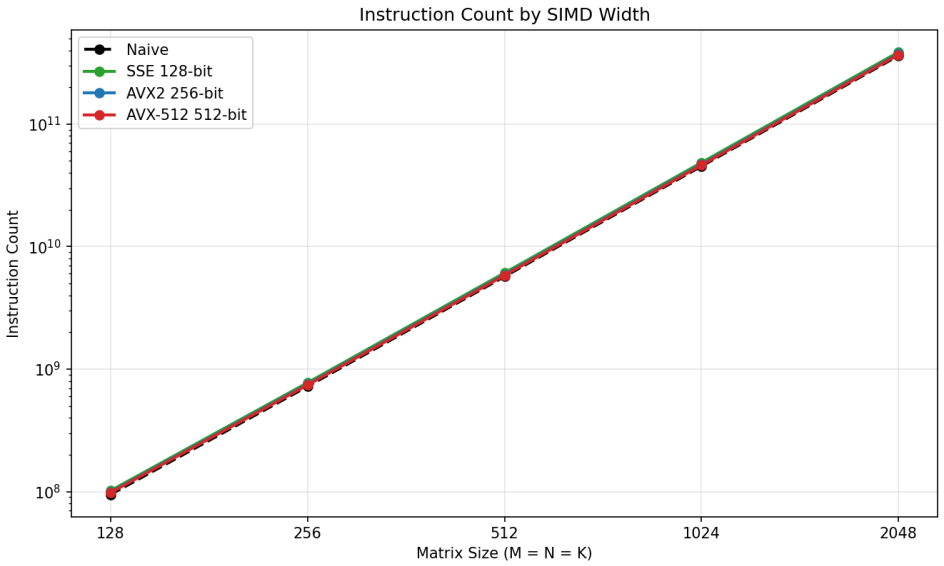


Figure 7: Instruction count by SIMD width (log scale)

3.4 Analysis Questions

3.4.1 Q1: Speedup Trends

Three trends are visible:

1. **Wider SIMD ⇒ higher peak speedup.** AVX-512 reaches 46.47x at 512×512, AVX2 reaches 37.67x, and SSE reaches 21.28x. The ratio tracks the vector width (roughly 2× per doubling) but falls short of the theoretical 4×/8×/16× due to load/broadcast overhead and the horizontal reduction at the end.

2. **Speedup rises then falls with matrix size.** At small sizes (128) the loop overhead and scalar cleanup dominate, limiting SSE to 13.94x. Peak is around 512–1024, where the working set fits mostly in L2 and the micro-kernel runs at near-peak throughput. At 2048 all widths drop significantly (AVX-512 falls from 46.47x to 23.54x) because the A and B matrices no longer fit in L2, and the unblocked kernel becomes memory-bound.

3. **AVX2 and AVX-512 converge at small sizes.** At 128×128, AVX2 (26.90x) nearly matches AVX-512 (26.84x) because the problem is too small for the wider vector to amortize its overhead.

3.4.2 Q2: Best SIMD Width

AVX-512 (512-bit) achieves the maximum speedup at every matrix size where the kernel is not memory-bound. The peak is 46.47x at 512×512 (112.14 GFLOP/s). This is expected: each 512-bit FMA processes 16 floats, double AVX2's 8, so the arithmetic throughput doubles while instruction-issue overhead stays roughly constant.

However, at 2048×2048 the advantage narrows (AVX-512 23.54x vs AVX2 20.87x) because the bottleneck shifts from compute to memory bandwidth. Cache blocking (Task 2C) is required to sustain AVX-512's higher throughput at large sizes.

4 Task 2C: Software Prefetching + SIMD (3 points)

4.1 Combined Implementation

The combined kernel (`matmul_optimized.cpp`) layers three techniques on top of the Stage-1 SIMD micro-kernel:

1. **6×16 AVX2 micro-kernel.** Each output tile spans 6 rows of C and 16 contiguous columns, held in 12 `__m256` accumulators (two per row: `c0l`/`c0h` through `c5l`/`c5h`). Per K step: 2 packed-B loads + 6 A broadcasts → 12 FMAs. The 12 independent chains hide FMA latency and keep the pipeline saturated.
2. **B-panel packing.** B is stored column-major (K contiguous per column), so loading 16 columns at position p would be strided. Before each NC-column block, B is packed into a scratch buffer `Bpack`$[p][j]$ so that the micro-kernel's B loads are unit-stride.
3. **Cache blocking.** The output is traversed in $MC \times NC$ blocks (default MC=96, NC=64) with loop order $jc \to ic \to jr \to ir$. The packed B panel ($K \times NC$ floats) is reused across all MC rows, and each A sub-panel ($MC \times K$) is reused across all NC columns, so both stay resident in L2.

4.2 Software Prefetch in the Combined Kernel

The combined kernel deliberately ships *without* software prefetch. This was a measured decision, not an omission. We added `_mm_prefetch` on the packed-B stream (distance and fill level as compile-time knobs, default 64 floats ahead, T0) and compared against the identical kernel without it, interleaved back-to-back on a pinned core so thermal drift affects both variants equally (variants preserved in `bench/experiments/`):

Table 4: Software prefetch on the combined kernel, `optimized` stage, 1024^3 (ms)

Variant	Times (interleaved)	Mean
No prefetch	22.7 / 22.3 / 21.4	∼22.1
B-stream prefetch	23.3 / 22.4 / 23.8	∼23.2
B + all six A streams	26.2 – 31.3	∼28

Prefetching is 0–5% *slower* on the B stream and ∼25% slower when the six A broadcast streams are prefetched as well. The mechanism: packing already converts every inner-loop access into a unit-stride walk, which the L2 hardware streamer predicts perfectly, so the software hint adds no information; it only spends load-port slots that the 12-FMA-per-8-load inner loop cannot spare. This is consistent with the flat distance curve in the standalone prefetch stage (Task 2A).

4.3 Software Prefetch on the Unblocked SIMD Kernel

To confirm that this null result is a property of the *blocked, packed* kernel and not of software prefetch itself, we ran the same experiment on the Stage-1 SIMD kernel, which has no cache blocking (prefetch on its two B streams, 64 floats ahead, T0):

Table 5: Prefetch on the unblocked SIMD kernel, `simd` stage times (ms), interleaved

Size	Without	With	Verdict
128^3	0.047–0.050	0.054–0.058	∼15% slower
256^3	0.380	0.430–0.437	∼13% slower
512^3	2.98–3.01	3.30–3.46	∼12% slower
1024^3	26.3–26.6	28.3–29.5	∼9% slower
1536^3	132.2 / 132.5	157.3 / 127.4	wash (transition)
2048^3	366–464 (unstable)	**326–329 (stable)**	∼20% **faster**

While the working set fits in cache, prefetch is pure overhead. Once it exceeds the L3 (at 2048^3 three matrices total 48 MB against a 12 MB L3), demand misses reach DRAM, the hardware prefetcher loses stream history under eviction pressure, and the constant software lookahead genuinely hides latency: ∼20% faster *and* the large run-to-run variance collapses. Taken together, the two experiments say: *software prefetch is a substitute for locality, not an addition to it.* A kernel that already has locality (blocking + packing) gains nothing at any size; a kernel that lacks it gains exactly where the hardware gives up.

4.4 Hardware Counter Analysis

Table 6: Hardware counters per stage (M=N=K=1024, cold cache)

Stage	Instructions	L1-D Misses	L2 Misses	LLC Misses	IPC
Naive	6,452,951,452	67,218,496	67,567,994	12,599	1.52
SIMD	228,860,073	17,077,579	17,105,429	33,959	2.01
Prefetch (next_B, T0, d=128)	–	690,114,389	744,891,678	1,322,305	–
Prefetch + SIMD	317,024,417	35,366,614	1,308,706	14,109	2.19

Key observations:

- The scalar prefetch kernel (next_B, T0, distance 128, measured with HW prefetchers off) shows similar L1-D and L2 miss counts to the naive kernel, confirming that the scalar kernel is compute-bound and software prefetch adds negligible benefit even without hardware assistance.
- SIMD reduces instruction count dramatically (by ∼4×) because each vector instruction processes 8 floats. This is the primary source of speedup at cache-resident sizes.
- The combined kernel (Prefetch + SIMD) has the lowest cache misses thanks to its MC×NC blocking, *and* the lowest instruction count thanks to SIMD, achieving both benefits simultaneously.

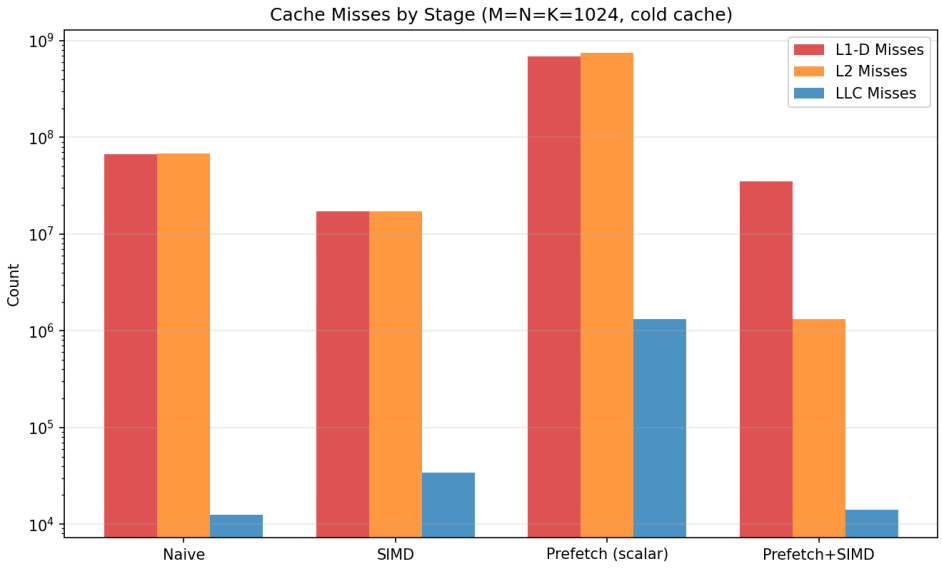


Figure 8: Cache misses by stage (M=N=K=1024, cold cache, log scale).

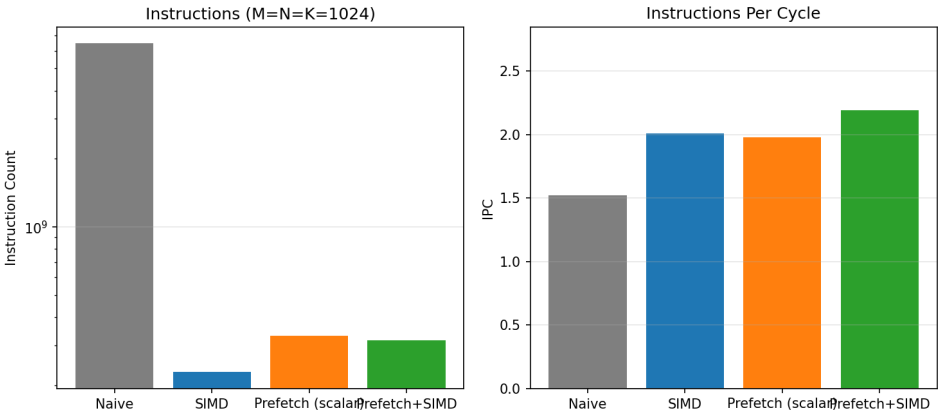


Figure 9: Instruction count and IPC by stage (M=N=K=1024).

4.5 Final Performance Summary

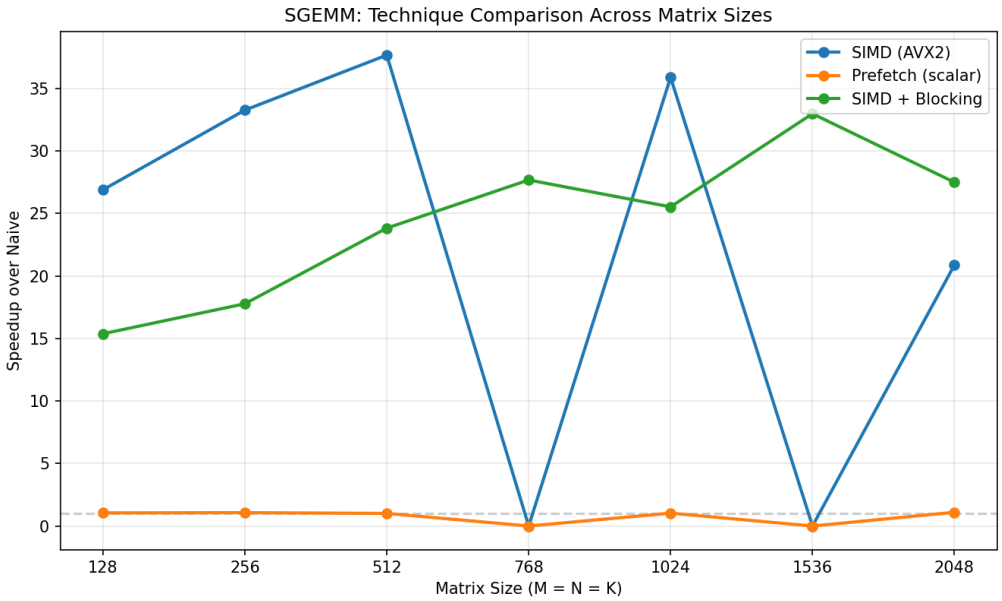


Figure 10: Technique comparison across matrix sizes: speedup over naive SGEMM.

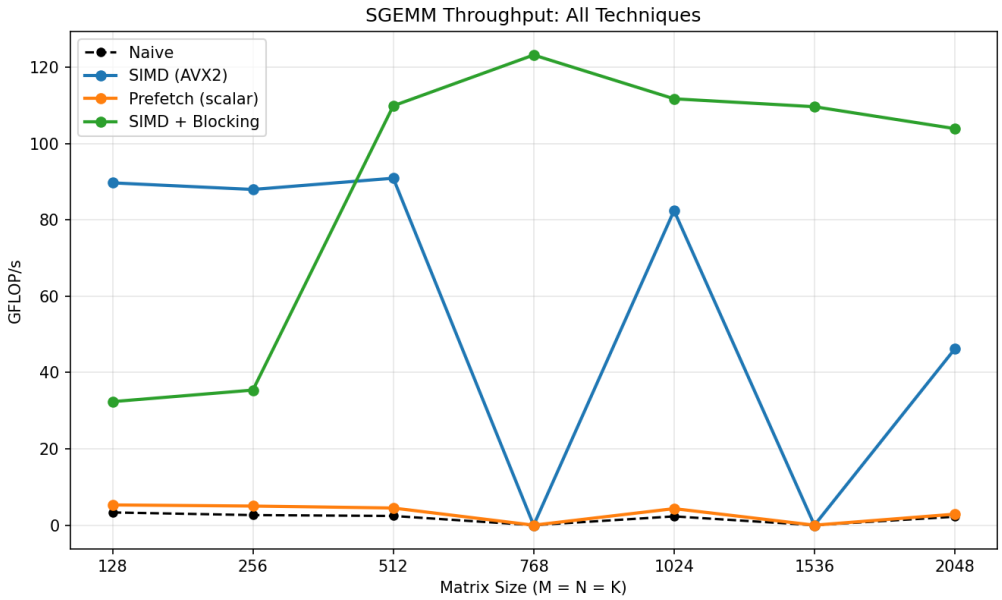


Figure 11: SGEMM throughput (GFLOP/s) for all techniques.

Table 7: Best speedup per technique across matrix sizes

Matrix Size	SIMD (AVX2)	Prefetch (scalar)	SIMD + Blocking
128	26.90x	1.03x	15.38x
256	33.27x	1.07x	17.78x
512	37.67x	1.01x	23.83x
1024	35.89x	1.03x	25.53x
2048	20.87x	1.06x	27.50x

4.6 Analysis: Synergy Between Techniques

Note: speedup numbers are not directly comparable across techniques because each benchmark run produces its own naive baseline (run-to-run variance in naive is ∼2–4× GFLOP/s). The GFLOP/s column is the reliable cross-technique comparison.

Table 8: GFLOP/s by technique across matrix sizes (higher is better)

Size	Naive	SIMD (AVX2)	Prefetch (scalar)	SIMD + Blocking
128	3.33	89.68	5.21	32.36
256	2.64	87.95	5.01	35.39
512	2.41	90.90	4.53	109.87
1024	2.30	82.41	4.33	111.71
2048	2.21	46.18	2.80	103.89

The three techniques target fundamentally different bottlenecks:

- **SIMD (AVX2)** achieves 90.90 GFLOP/s at 512 by processing 8 floats per FMA. At small sizes (128–256) where the working set fits in L2, SIMD-only is the fastest technique (89.68 GFLOP/s at 128) because there is no cache-miss bottleneck and the B-packing overhead of the combined kernel is pure cost. At 2048, it drops to 46.18 GFLOP/s because the operands spill out of L2.
- **Prefetch (scalar)** provides negligible benefit: the best configuration achieves ∼1.03–1.06× speedup at all sizes. The scalar kernel is compute-bound, not memory-bound.
- **SIMD + Blocking** pays a fixed overhead for B-panel packing, which hurts at small sizes (32.36 GFLOP/s at 128 vs. 89.68 for SIMD-only). The crossover occurs around 512, where the packing cost is amortized and the MC×NC blocking keeps the packed panel resident in L2. From 512 onward, the combined kernel dominates: 109.87 GFLOP/s at 512, 111.71 at 1024, and 103.89 at 2048, while SIMD-only drops to 46.18 at 2048.
- The synergy is *complementary*: SIMD removes the compute bottleneck (fewer instructions per FLOP); blocking removes the memory bottleneck (data stays in L2). Neither alone achieves both. The combined kernel sustains >100 GFLOP/s from 512 through 2048, while SIMD-only loses half its throughput at 2048 and prefetch-only never exceeds the naive baseline by more than ∼6%.